



SDN Load Balancer

CS 3001 - Computer Networks

Member:

Muhammad Umer Fazal

Proposed Project Description

The Proposed system is to design and implement the Software Defined Networking (SDN) based dynamic load balancer using POX controller and Mininet network emulator. In traditional networks, we used hardware based load balancers distribute network or application traffic evenly among multiple backend servers. But these hardware's are expensive, difficult to configure and not flexible for on-demand applications. SDN introduced the new paradigm in networking architectural approach where the control plane logically centralized or logically separated from data forwarding devices.

An SDN controller will be used to manage the distribution of traffic among several backend servers. The controller will function as a centralized decision making entity that observes client requests entering the network and then decides on the best server to send the request. Mininet will be employed to emulate the network, as it is capable of creating virtual hosts, switches and links to model a real world network.

The system is made up of a single OpenFlow switch, several server nodes and numerous client nodes. Clients will be making requests to a virtual service IP address (VIP) which represents the load balanced service. When the request enters the switch, it will forward the packet to the POX controller. The controller will inspect the request and decide, based on a load balancing algorithm, which server should process it. This is done only for the first packet of each connection. After this, flow rules will have been installed in our switch by controller so that packets of other direction in this connection are directly forwarded by switch without involving controller.

The project will implement two widely used load balancing algorithms:

- Round Robin Algorithm: In this scheme, the scheduler distributes clients' requests or connection to all servers in rotation i.e. one by one in circular order till all connections are used up.
- Least Connection Algorithm: Requests are sent to the server with least number of active connections.

This will also have a server health monitoring in the system will be done using ARP probing by periodically sending the ARP request by controller to the servers to find whether they are active/reachable or unreachable. If a server doesn't respond then it will be eliminated from list of available servers so that traffic won't be forwarded to failed node.

To mimic real services, all servers will run a small HTTP web server. Clients will make requests to the service using the virtual IP address, which will be load balanced by the backend servers in NAT mode.

Another component of the system is a simple DNS server. This server resolves domain queries to the virtual service IP address. Therefore a client can access the service using a domain name rather than directly using an IP address.

Overall, the system can be taken as an example to show that SDN allows implementing flexible, programmable and efficient network services such as load balancing.

Functional Features

The following functional features will be implemented in the proposed project:

1. **Dynamic Load Balancing**
 - Incoming client requests will be distributed among multiple servers dynamically.
2. **Round Robin Load Balancing Algorithm**
 - Requests will be sequentially distributed across servers.
3. **Least Connection Load Balancing Algorithm**
 - Requests will be forwarded to the server with the fewest active connections.
4. **Server Health Monitoring**
 - The controller will periodically check server availability using ARP probes.
5. **Flow Rule Installation**
 - The controller will install OpenFlow rules in the switch to handle traffic forwarding efficiently.
6. **Session Persistence**
 - Once a connection is established with a specific server, subsequent packets will continue to use the same server.
7. **DNS Resolution for Virtual Service**
 - A simple DNS service will resolve domain queries to the load balancer's virtual IP.
8. **Network Topology Simulation**
 - A network topology containing multiple clients, servers, and an OpenFlow switch will be simulated using Mininet.
9. **HTTP Service Simulation**
 - Each server will run a lightweight web service to respond to client requests.

Plan of Work (Next 5 Weeks)

The project will be completed over the next five weeks.

Week 1 – Research and Environment Setup

- Study the architecture of **Software Defined Networking (SDN)** and the **OpenFlow protocol**.
- Install and configure **Mininet network emulator**.
- Design the initial **network topology** including one switch, multiple clients, and server nodes.
- Install and configure the **POX SDN controller**.
- Study how POX interacts with OpenFlow switches.
- Establish the connection between the **POX controller and the Mininet switch**.

Week 2 – Network Topology Implementation

- Implement the **Mininet topology** containing multiple client hosts and server hosts.
- Configure IP addressing and connectivity between all hosts.
- Verify connectivity using **ping tests** and ensure the switch correctly connects all nodes.
- Implement the **basic packet handling logic in the POX controller**.
- Configure the controller to receive and process **PacketIn messages** from the switch.
- Test communication between the switch and controller.

Week 3 – Load Balancing Algorithm Implementation

- Implement the **Round Robin load balancing algorithm** in the controller.
- Configure backend servers to run **HTTP web services**.
- Generate client requests and test the load distribution across servers.
- Implement the **Least Connection load balancing algorithm**.
- Implement **connection tracking** to monitor active sessions on each server.
- Compare the traffic distribution behavior between algorithms.

Week 4 – Traffic Management and Flow Control

- Implement **OpenFlow flow rule installation** in the switch.
- Configure packet rewriting to forward client requests to selected backend servers.
- Implement **session persistence** so that packets belonging to the same connection are forwarded to the same server.
- Implement **server health monitoring using ARP probing**.
- Detect unavailable servers and dynamically remove them from the active server pool.
- Update the controller logic to avoid forwarding traffic to failed servers.

Week 5 – DNS Integration and Final Testing

- Perform **system integration testing** with multiple clients sending requests simultaneously.
- Analyze traffic distribution results and verify correct load balancing behavior.
- Assist in debugging and performance optimization.
- Implement a **simple DNS server** to resolve domain requests to the virtual service IP address.
- Integrate DNS functionality with the load balancing system.
- Perform **final system evaluation and debugging**.

References

1. McKeown, N., et al. “OpenFlow: Enabling Innovation in Campus Networks.” ACM SIGCOMM Computer Communication Review, 2008.
2. Open Networking Foundation.
<https://opennetworking.org>
3. POX Controller Documentation
<https://noxrepo.github.io/pox-doc/html/>
4. Mininet Network Emulator Documentation
<http://mininet.org/documentation/>
5. Feamster, N., Rexford, J., & Zegura, E. “The Road to SDN: An Intellectual History of Programmable Networks.” ACM SIGCOMM CCR.